

SECURE MULTI-PARTY COMPUTATION (MPC) VOTING SYSTEM USING SHAMIR SECRET SHARING

PROJECT REPORT

**Submitted for Project in B.Tech - [Computer Science and Engineering
AI ML] for**

BCSE309L - Cryptography and Network Security

By

Parun Sethupathy 23BAI1576

Aayush Sundaresan 23BAI1544

Slot - A1

Name of faculty: Dr. Sellam V.

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Winter Semester 2025-26

ABSTRACT

Digital voting systems traditionally rely on a central, trusted authority to collect, tally, and store votes, creating a single point of failure for data breaches or insider manipulation. This project implements a privacy-preserving e-voting system using Multi Party Computation (MPC) and Shamir Secret Sharing. By shifting the cryptographic workload to the client side, votes are encoded into vectors and split into meaningless shares before ever leaving the user's device. These shares are distributed across independent servers that compute partial sums without ever decrypting the data. The final tally is reconstructed securely using Lagrange interpolation. This report details the distributed architecture, the cryptographic methodology, and the observed limitations of deploying this multi server environment on local hardware.

INTRODUCTION

Motivation

The convenience of electronic voting is heavily outweighed by severe security concerns. If a central database stores votes even in encrypted form the system remains vulnerable if the decryption keys are compromised or if the database administrator acts maliciously. The motivation behind this project is to build a trustless digital democracy; a system where we do not have to trust any single server, database, or administrator with our actual vote data.

Problem Statement

How can a system accurately tally votes without any single entity ever knowing who voted for whom? The goal is to design a distributed web application where user authentication is decoupled from vote storage, and where the mathematical aggregation of votes happens entirely on fragmented, unreadable data shares.

SYSTEM DESIGN

Proposed System

The system abandons traditional database storage for votes. Instead, it uses a distributed 5-node architecture (Frontend, Backend, and 3 independent MPC Servers).

Vote Encoding: Candidates are mapped to a vector array (e.g., voting for Candidate A results in $[1, 0, 0]$).

Share Distribution: The vector is split into three shares using a random polynomial.

Database Role: An SQLite database is strictly used to maintain user credentials and a boolean `has_voted` flag to prevent double voting. It absolutely does not store vote preferences.

Architecture

Frontend (React): Handles UI, vector encoding, and client-side Shamir Secret Sharing.

Backend (Flask): Manages user authentication and audit logging.

MPC Servers (Flask): Three separate servers (running on ports 6001, 6002, 6003). Each receives only one share of the vote and computes a running local sum.

Reconstruction Server: Fetches the aggregated sums from the three MPC servers and applies Lagrange interpolation to produce the final human-readable vote counts.

METHODOLOGY

Shamir Secret Sharing

The core cryptographic engine relies on polynomial interpolation. The secret (the vote vector s) is hidden as the y-intercept of a random polynomial: $f(x) = s + a_1 \cdot x \bmod p$ (where p is a large prime number and a_1 is a random coefficient). The frontend generates three points on this curve and sends one point to each MPC server. Because it requires multiple points to reconstruct a line/curve, a single server holding one point learns zero information about the original vote.

Homomorphic Aggregation via MPC

A key mathematical property of Shamir Secret Sharing is that it is additively homomorphic. This means that if Server 1 adds all of its $f(1)$ shares together, and Server 2 adds all its $f(2)$ shares together, the resulting sums can be interpolated to find the sum of the original secrets. The servers effectively tally the votes without ever knowing what the votes are.

IMPLEMENTATION & RESULTS

System Execution

The system was successfully implemented using React for the frontend and Flask for the microservices. During a live test scenario (e.g., Alice votes for Candidate 1, Bob for Candidate 2), the frontend seamlessly intercepted the form submission, generated the mathematical shares, and routed them to the three distinct local ports.

Verification and Visualization

The Reconstruction Server successfully compiled the partial sums and accurately derived the final vote count (e.g., Candidate 1 = 2 votes, Candidate 2 = 1 vote). These results were securely passed back to the React frontend and visualized in real-time using Chart.js (Bar and Pie charts), proving that the distributed mathematical pipeline was functional end-to-end.

PERFORMANCE ANALYSIS & LIMITATIONS

While the cryptographic theory proved robust, the practical implementation on a local development environment highlighted several critical engineering constraints:

Zero Fault Tolerance: Because our MPC implementation requires all shares for the current reconstruction logic, if even one of the three MPC Flask servers crashes or goes offline, the entire tallying process fails.

Concurrency Bottlenecks: SQLite was chosen as a lightweight development database. However, it struggles with concurrent write locks. If scaled, the “has_voted” status updates would require a more robust database like PostgreSQL.

Network Security: For this local build, HTTP was used. In a real-world scenario, sending shares over unencrypted HTTP would expose the polynomial points to packet sniffing, necessitating strict HTTPS/TLS enforcement.

CONCLUSION & FUTURE SCOPE

Conclusion

This project successfully bridges the gap between abstract cryptographic theory and a practical web application. By enforcing client-side secret sharing and multi-server aggregation, we proved that it is entirely possible to build a digital voting system that guarantees information-theoretic privacy for the voter while maintaining mathematical accuracy for the election administrators.

Future Scope

To elevate this project from a local prototype to a production-ready system, future iterations must include:

- Server Fault Tolerance: Implementing a (k, n) threshold system (e.g., needing only 2 out of 3 servers to reconstruct the tally) to prevent system halting.
- Containerization: Using Docker to isolate the MPC servers, simulating a true distributed network rather than relying on localhost ports.
- Malicious Server Detection: Integrating Zero-Knowledge Proofs (ZKPs) to verify that a client hasn't sent a malformed vector (e.g., $[5, 0, 0]$ to cast 5 votes at once) without revealing the vector itself.

8. REFERENCES

1. Shamir, Adi. "How to share a secret." *Communications of the ACM* 22.11 (1979): 612-613.
2. Cramer, R., Damgård, I., & Nielsen, J. B. (2015). *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press.
3. Flask Documentation: <https://flask.palletsprojects.com>